

CCF CSP 300+

CCF CSP Certified Software Professional 300 500

GitHub —

GitHub

README GitHub

README.md	CCF CSP 300+
ALGORITHMS.md	—23+C++
PAST_EXAMS.md	—7 CCF CSP+C++

README main

GitHub branch

- main Branch
- copilot/add-csp-certification-solution
- ALGORITHMS.md PAST_EXAMS.md

Pull Request

- Pull requests
- CCF CSP ALGORITHMS PR
- Files changed
- Merge pull request main



- [ALGORITHMS.md](#) 23 ★
- [PAST_EXAMS.md](#) 2020–2022 CCF CSP C++ ★

PDF

- [README.pdf](#)
- [ALGORITHMS.pdf](#)
- [PAST_EXAMS.pdf](#)

- [1.](#)
- [2. 300+](#)
- [3.](#)
- [4.](#)
- [5.](#)
- [6. 8](#)
- [7.](#)
- [8.](#)
- [9.](#)

1.

	CCF
	OJ
	5
	100
	500
	4
	C/C++JavaPython
	3 3912
	https://www.ccf.org.cn/csp/

2-4

	CSP	
	★★★★★	
	★★★★	
	★★★★★	
map/set	★★★★★	
	★★★★★	
	★★★★	
	★★★★	
	★★★	

4-6

	CSP	
	★★★★★	
	★★★★★	
	★★★★	
DFS	★★★★★	
BFS	★★★★★	
DP	★★★★★	
	★★★★	
KMP/Trie	★★★	

6-8

- CSP 1-3 4
-
- 4

4. □□□□□□□□

□ 1 □□□□□100□□

□□□ □□□□□□□□□□□□□□

□□□

- □□□□□□□100% □□□□□
- □□□□□□□□□□□□□□
- □□□□□□□□□□□□□□
- □□□□□□□□ 20 □□

□□□□□

- □□□□
- □□□□
- □□□□/□□
- □□□□□/□□

□ 2 □□□□□100□□

□□□ □□□□□□□□□□□□

□□□

- □□□□□□□□□□□□□□□□
- □□ map/set □□□□
- □□□□□□□□□ int □□□□□ long long□
- □□□□□□□□ 30 □□

□□□□□

- □□□□□□□
- □□□□□
- □□□□□
- □□□□□

□ 3 □□□□□80–100□□

□□□ □□□□□□□□□□□□

□□□

- □□□□□□□□□□□□□□□□BFS/DFS/DP/□□□
- □□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□ 50–60 □
- □□□□□□□□ 60 □□

□□□□□

- □□□□□□□□□□□□□□□□BFS/DFS□
- □□□□□□□
- □□□□□□□
- □/□□□□□

第 4 章 面试准备30-60分钟

面试准备时间

面试

- 面试准备时间
- 面试准备时间50%-60%
- 面试准备时间
- 面试准备时间

面试准备

- 面试准备时间DP
- 面试准备时间
- 面试准备时间

第 5 章 面试准备0-30分钟

面试准备时间

面试

- 面试准备时间
- 面试/面试准备
- 面试准备时间
- 面试准备时间

5. 面试准备

面试准备

面试	面试	面试
面试Luogu	https://www.luogu.com.cn	CSP 面试准备
CSP 面试	https://csp.thussat.com	面试准备
LeetCode面试	https://leetcode.cn	面试准备
Codeforces	https://codeforces.com	面试准备Div.3-Div.2
AcWing	https://www.acwing.com	面试准备

面试准备

面试	面试	面试
面试准备	面试	面试-面试

姓名	姓名	姓名
姓名4	Sedgewick	姓名
姓名	姓名	姓名-姓名
姓名	CLRS	姓名

- 姓名
- **AcWing** 姓名 CSP 姓名
- 姓名
- **B CSP** 姓名"CSP姓名"姓名

6. 姓名8 姓名

姓名 1-2 姓名

- [] 姓名
- [] 姓名 20+ 姓名
- [] 姓名 20+ 姓名
- [] 姓名 CSP 姓名 1 姓名
- [] 姓名CSP 姓名 1 姓名

姓名 3-4 姓名

- [] 姓名 STL/姓名
- [] 姓名 30+ 姓名
- [] 姓名 20+ 姓名
- [] 姓名 CSP 姓名 2 姓名
- [] 姓名CSP 姓名 2 姓名 80+ 姓名

姓名 5-6 姓名

- [] 姓名 BFS/DFS 姓名
- [] 姓名DP姓名
- [] 姓名
- [] 姓名 CSP 姓名 3 姓名
- [] 姓名CSP 姓名 3 姓名 60+ 姓名

姓名 7 姓名

- [] 姓名 3 姓名 20+ 姓名
- [] 姓名 4 姓名

- [] 00000000BFS0000000000
- [] 000CSP 3 000 80+ 000 4 00 30 0

8 00000000

- [] 2 00000000 CSP 000000 4 0000
- [] 00000000000000
- [] 000000000000000000
- [] 00000000 300+ 0

7. 00000000000000

7.1 00000000

```
// 000000
vector<long long> prefix(n + 1, 0);
for (int i = 1; i <= n; i++)
    prefix[i] = prefix[i - 1] + a[i];

// 00 [l, r] 00
long long sum = prefix[r] - prefix[l - 1];
```

7.2 0000

```
// 000000 >= target 000
int lo = 0, hi = n;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < target) lo = mid + 1;
    else hi = mid;
}
// lo 00000000 lower_bound(a, a+n, target) - a
```

7.3 BFS0000/000000

```
#include <queue>
using namespace std;

void bfs(int start, vector<vector<int>>& graph, int n) {
    vector<int> dist(n, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : graph[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```



```

    }
  }
}

```

7.4 DFS □□□□/□□□

```

vector<bool> visited(n, false);

void dfs(int u, vector<vector<int>>& graph) {
    visited[u] = true;
    for (int v : graph[u]) {
        if (!visited[v]) {
            dfs(v, graph);
        }
    }
}

```

7.5 □□□

```

vector<int> parent, rank_;

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]); // □□□□
    return parent[x];
}

void unite(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return;
    if (rank_[px] < rank_[py]) swap(px, py);
    parent[py] = px;
    if (rank_[px] == rank_[py]) rank_[px]++;
}

// □□□
void init(int n) {
    parent.resize(n);
    rank_.assign(n, 0);
    iota(parent.begin(), parent.end(), 0);
}

```

7.6 Dijkstra □□□

```

#include <queue>
using namespace std;
typedef pair<long long, int> pli;

vector<long long> dijkstra(int src, int n,
    vector<vector<pair<int, long long>>>& adj) {
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pli, vector<pli>, greater<pli>> pq;

```

```

dist[src] = 0;
pq.push({0, src});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;
    for (auto [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}
return dist;
}

```

7.7 `01`

```

// W n w[i] v[i]
vector<long long> dp(W + 1, 0);
for (int i = 0; i < n; i++) {
    for (int j = W; j >= w[i]; j--) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
// dp[W]

```

7.8 `sort + lambda`

```

#include <algorithm>
// 
sort(arr.begin(), arr.end(), [](const auto& a, const auto& b) {
    if (a.first != b.first) return a.first > b.first;
    return a.second < b.second;
});

```

7.9

```

// 
const long long MOD = 1e9 + 7, BASE = 131;
int n = s.size();
vector<long long> h(n + 1, 0), pw(n + 1, 1);
for (int i = 0; i < n; i++) {
    h[i + 1] = (h[i] * BASE + s[i]) % MOD;
    pw[i + 1] = pw[i] * BASE % MOD;
}
// [l, r] 0-indexed
auto getHash = [&](int l, int r) {
    return (h[r + 1] - h[l] * pw[r - l + 1] % MOD + MOD * 2) % MOD;
};

```

7.10

```
// 下一个更大数
vector<int> nextGreater(n, n); // 返回 n 的下一个更大数
stack<int> stk;
for (int i = 0; i < n; i++) {
    while (!stk.empty() && a[stk.top()] < a[i]) {
        nextGreater[stk.top()] = i;
        stk.pop();
    }
    stk.push(i);
}
```

8. 下一个更大数

下一个更大数

数字	下一个更大数	下一个更大数
10	10 的下一个更大数是 15	10 的下一个更大数是 15
1 2	20 的下一个更大数是 30	20 的下一个更大数是 30
2 3	30 的下一个更大数是 60	30 的下一个更大数是 60
3 4	60 的下一个更大数是 60	60 的下一个更大数是 60
4 5	60 的下一个更大数是 60	60 的下一个更大数是 60
5	60 的下一个更大数是 60	60 的下一个更大数是 60

下一个更大数

1. 编译选项 C++11

```
cpp ios::sync_with_stdio(false); cin.tie(nullptr);
```

2. 编译选项 long long 类型 __int128 类型 __int128 在 GCC/Clang 中支持，C++ 中 MSVC 不支持 CSP 在 GCC 中支持

3. 编译选项

- 编译选项
- 编译选项
- n=0 或 n=1 编译选项

4. 编译选项

- 编译选项
- 编译选项 cerr 编译选项

- 5. 字符串
 - 字符串匹配
 - 字符串排序
 - 字符串统计
- 6. 图
 - 图的遍历
 - 最短路径
 - 拓扑排序

9. 面试准备

知识点	备注
字符串	字符串 <code>long long</code> 存储
字符串	字符串+10进制字符串
字符串	字符串统计
字符串	字符串排序
字符串TLE	字符串 $O(n^2)$ 字符串统计 I/O
字符串MLE	字符串统计
字符串	字符串统计 <code>eps = 1e-9</code>
字符串	字符串统计

300+ 面试

字符串 + STL

↓

1200002000 — 50-100

↓

3000060-100 — BFS/DFS/DP 200

↓

4000030-60 — $O(n^2)$ $O(n^3)$

↓

290-360  300

□□□□□□□□□□ + □□□□ + □□□□□ □□ 8 □□□□□□300+ □□□□□□□□

CCF CSP